20/24 points + 0.75 bonus
=> Grade: 8.6

NURa Hand-in 1

Silvan Toet (s1827332)

March 5, 2026

1 Poisson Distribution

In this question, we are asked to compute the Poisson distribution for integer k:

$$P_\lambda(k) = \frac{\lambda^k e^{-\lambda}}{k!} \tag{1}$$

given a positive mean λ. We are asked to compute $P_\lambda(k)$ for the values in 1. We are restricted to 32 bit numbers. This means we will get overflow errors for most combinations of k and λ due to the rapid growth of $k!$ and λ^k and underflow due to $e^{-\lambda}$. To prevent this we rewrite the equation in log10 form:

$$\tag{2}$$

$$log_{10}(P_\lambda(k)) = log_{10}(\lambda^k) + log_{10}(e^{-\lambda}) - log_{10}(k!) \tag{3}$$

$$= k\ log_{10}(\lambda) - \lambda\ log_{10}(e) - log_{10}(\prod_{i=1}^{k} i) \tag{4}$$

$$= k\ log_{10}(\lambda) - \lambda\ log_{10}(e) - \sum_{i=1}^{k} i\ log_{10} \tag{5}$$

$$-> P_\lambda(k) = 10^{(k\ log_{10}(\lambda) - \lambda\ log_{10}(e) - \sum_{i=1}^{k} i\ log_{10})} \tag{6}$$

We choose log10 over the natural logarithm as this leads to a larger 'compression' of the large numbers, preventing overflow for a wider input range.

The full code used for this question:

```python
from math import e

import numpy as np


def Poisson(k: np.int32, lmbda: np.float32) -> np.float32:
    """Calculate the Poisson probability for k occurrences with mean lmbda.
    Parameters:
        k (np.int32): The number of occurrences.
        lmbda (np.float32): The mean number of occurrences.
    Returns:
        np.float32: The probability of observing k occurrences given the mean lmbda.
    Since the calculation will quickly overflow due to the extremely large numbers involved, we rewrite the equation in log10 space.
```

```python
    Then we return 10^<log version>. For the derivation see the Latex document.
    """
    return 10 ** (
        k * np.log10(lmbda)
        - lmbda * np.log10(e)
        - sum(np.log10(i) for i in range(1, k + 1))
    )


def main() -> None:
    # (lambda, k) pairs:
    values = [
        (np.float32(1.0), np.int32(0)),
        (np.float32(5.0), np.int32(10)),
        (np.float32(3.0), np.int32(21)),
        (np.float32(2.6), np.int32(40)),
        (np.float32(100.0), np.int32(5)),
        (np.float32(101.0), np.int32(200)),
    ]
    with open("Poisson_output.txt", "w") as file:
        for i, (lmbda, k) in enumerate(values):
            P = Poisson(k, lmbda)
            if i < len(values) - 1:
                file.write(f"{lmbda:.1f} & {k} & {P:.6e} \\\\ \\hline \n")
            else:
                file.write(f"{lmbda:.1f} & {k} & {P:.6e} \n")


if __name__ == "__main__":
    main()
```

The results of $P_\lambda(k)$ for selected values of k and λ are shown in Table 1.

Table 1: Poisson probability distribution for selected k and λ.

λ	k	$P_\lambda(k)$
1.0	0	3.678794e-01
5.0	10	1.813280e-02
3.0	21	1.019339e-11
2.6	40	3.615124e-33
100.0	5	3.100063e-36
101.0	200	1.269510e-18

2 Vandermonde Matrix and Interpolation

For question 2 we are asked to solve the equation $\boldsymbol{Vc} = \boldsymbol{y}$ for c. This gives us the coeffients of the unique polynomial that passes through all points:

$$y_i = \sum_{j=0}^{N-1} c_j x_i^j (i = 0, ..., N-1) \tag{7}$$

First we construct the Vandermonde matrix V using x values in the supplied datafile. Then in 2a we first create the LU decomposition of V with Crout's algorithm. Next we solve for the supplied y values to find the coefficients c of the unique polynomial. In 2b we use polynomial interpolation

using Neville's algorithm to compare our results. In 2c we use iterative improvement of the LU decomposition to get a better estimate. In 2d we time the different methods. After printing all our code we will discuss our results.

2.1 (2a) LU decomposition

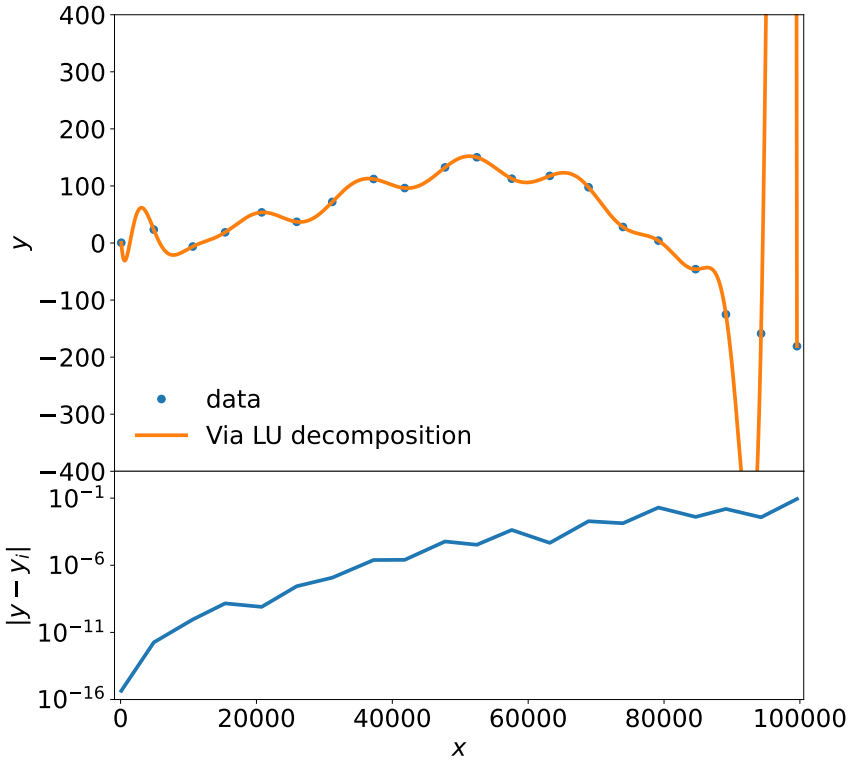

Figure 1: Polynomial fit evaluated using LU decomposition. Top: data points and interpolated curve. Bottom: absolute error at the data points on a log scale.

2a: 7.25/8 + 0.25

2.2 (2b) Neville's algorithm

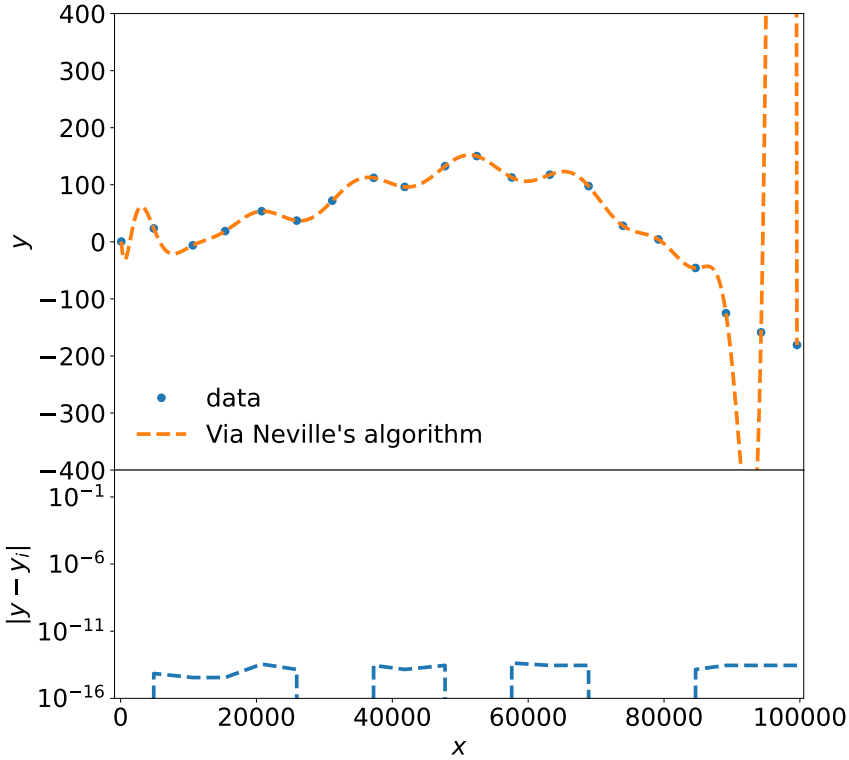

Figure 2: Interpolation using Neville's algorithm. Top: data points and interpolated curve. Bottom: absolute error at the data points on a log scale.

2.3 (2c) Improving the LU decomposition

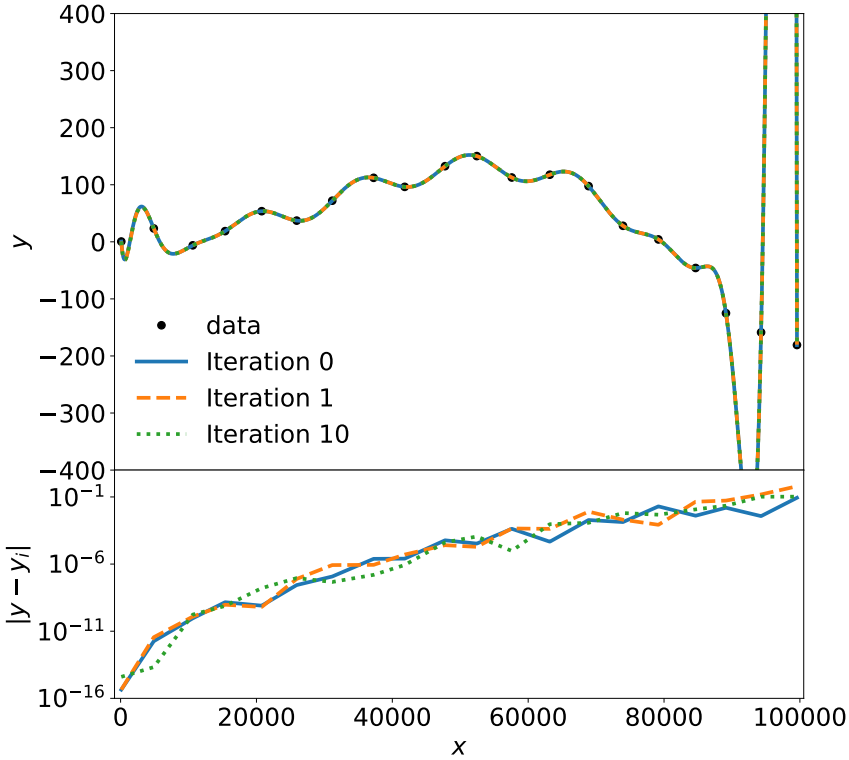

Figure 3: LU-based solution with iterative refinement (showing iterations 0, 1 and 10). Top: interpolated curves. Bottom: absolute error at the data points on a log scale.

2.4 (2d) Timing

Timing results (average per run):

- Execution time for part (a): 0.00164 seconds
- Execution time for part (b): 0.20940 seconds
- Execution time for part (c): 0.00358 seconds

2.5 Code for Question 2

The following code was used for parts (2a)–(2d):

```
from copy import deepcopy
import os
import sys
import timeit
```

```python

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np

mpl.rcParams["font.size"] = 20
mpl.rcParams["axes.labelsize"] = 20
mpl.rcParams["xtick.labelsize"] = 20
mpl.rcParams["ytick.labelsize"] = 20


def load_data():
    """Function to load the data from Vandermonde.txt.

    Returns
    -------
    x (np.ndarray): Array of x data points.

    y (np.ndarray): Array of y data points.
    """
    data = np.genfromtxt(
        os.path.join(sys.path[0], "Vandermonde.txt"), comments="#", dtype=np.float64
    )
    x = data[:, 0]
    y = data[:, 1]
    return x, y


def construct_vandermonde_matrix(x: np.ndarray) -> np.ndarray:
    """Construct the Vandermonde matrix V with V[i,j] = x[i]^j.

    Parameters
    ----------
    x : np.ndarray, x-values.

    Returns
    -------
    V : np.ndarray, Vandermonde matrix.
    """
    V = np.zeros((len(x), len(x)), dtype=np.float64)
    for i, xvalue in enumerate(x):
        for j in range(len(x)):
            V[i, j] = xvalue**j
    return V


class LUDecomposition:
    def __init__(self, a: np.ndarray):
        """This class performs LU decomposition, where a matrix A is decomposed in two matrices L and U.
        Here L is a Lower Triangular matrix (only elements on or below diagonal), and U is an Upper triangular matrix (only elements on or above the diagonal).
        We can use these to solve the equation A*x = (L*U)*x = L*(U*x) = b, by first solving L*y=b and then U*x=y.
        Use the constructor of the class to do the decomposition, using Crout's algorithm (slide 12 lecture 3)
        Then the instance of the object holds the LU decomposed matrix.
        We can then call the solve method as many times as we like, for as many b's as we like.
        """
```

```python
        self.a = a  # save reference to a for use in the .improve() method for iterative improvement based on paragraph 2.5
        self.nrows, self.ncolumns = a.shape[0], a.shape[1]
        self.LU_matrix = np.identity(
            self.ncolumns
        )  # step 1: construct an identity matrix to start
        for j in range(self.ncolumns):
            self.LU_matrix[0, j] = a[0, j]  # for i = 0, beta_0j = a_0j
            for i in range(
                1, self.nrows
            ):  # starting from 0 or 1 should not make a difference, as the first row is already set to the correct values, but we can skip it to save some time
                if i <= j:
                    self.LU_matrix[i, j] = a[i, j] - sum(
                        self.LU_matrix[i, k] * self.LU_matrix[k, j] for k in range(i)
                    )  # beta_ij = a_ij - sum (alpha_ik beta_kj) from k=0 to i-1. range doesn't include end point!
                if i > j:
                    self.LU_matrix[i, j] = (
                        a[i, j]
                        - sum(
                            self.LU_matrix[i, k] * self.LU_matrix[k, j]
                            for k in range(j)
                        )
                    ) / self.LU_matrix[
                        j, j
                    ]  # alpha_ij = a_ij - sum (alpha_ik beta_kj) from k=0 to j-1. range doesn't include end point!

    def get_LU_decomposition(self):
        return self.LU_matrix

    def solve(self, b: np.ndarray) -> np.ndarray:
        # Forward substitution first: L*y=b
        b_size = len(b)
        y = np.zeros(b_size)
        x = np.zeros(b_size)
        y[0] = b[
            0
        ]  # don't need to divide by LU_matrix[i,i] as for the forward substitution, the diagonal elements of L are all 1's
        for i in range(1, b_size):
            y[i] = b[i] - sum(self.LU_matrix[i, j] * y[j] for j in range(i))

        # Now do the backsubstitution
        x[b_size - 1] = y[b_size - 1] / self.LU_matrix[b_size - 1, b_size - 1]
        for i in reversed(range(b_size)):
            x[i] = (
                y[i] - sum(self.LU_matrix[i, j] * x[j] for j in range(i + 1, b_size))
            ) / self.LU_matrix[i, i]
        return x

    def iterative_solve(self, b: np.ndarray, iterations: int):
        """Improve the solution of A*x=b by iteratively solving A*delta = r, where r is the residual of the previous solution, and adding delta_x to the previous solution. Do this ntimes times.
        b is the right hand side. x is the initial solution.
        """
        x = self.solve(b)  # initial solution
        coeffs_per_iteration = [np.zeros_like(x) for _ in range(iterations + 1)]
```

```python
        coeffs_per_iteration[0] = deepcopy(x)
        for i in range(iterations):
            r = b - self.a @ x  # compute the residual r (delta_b)
            delta_x = self.solve(r)  # solve A * delta_x = delta_b
            x += delta_x  # update the solution
            coeffs_per_iteration[i + 1] = deepcopy(x)
        return coeffs_per_iteration


def evaluate_polynomial(c: np.ndarray, x_eval: np.ndarray) -> np.ndarray:
    """
    Evaluate y(x) = sum_j c[j] * x^j.

    Parameters
    ----------
    c : np.ndarray
        Polynomial coefficients.
    x_eval : np.ndarray
        Evaluation points.

    Returns
    -------
    y_eval : np.ndarray
        Polynomial values.
    """
    # evaluate the polynomial at x_eval using the coefficients c from vandermonde_solve_coefficients
    # evaluate efficiently using the formula on page 202 of the book "Numerical Recipes"
    polynomial_at_evaluated_points = np.zeros_like(x_eval)
    for i, x in enumerate(x_eval):
        p = c[-1]
        for j in reversed(range(len(c) - 1)):
            p = p * x + c[j]
        polynomial_at_evaluated_points[i] = p
    return polynomial_at_evaluated_points


class BaseInterpolater:
    """This class is a base class to do general interpolation tasks, such as bisection and testing if xdata is monotonic.
    In the tutorial I used this base class as a superclass for both the LinearInterpolator and PolynomialInterpolater classes.
    In this Python file I have only placed the base and polynomial interpolator classes.
    """

    def find_starting_index_and_closest_index(
        self, x: float, xdata: np.ndarray, m: int
    ) -> tuple[int, int]:
        """X is the value to interpolate the function f at, aka we want to know f(x).
        xdata are the x values of the measured data points.
        m is the order of the interpolation, aka m=2 linear interpolation.
        we want to find the starting index of the points we want to use for interpolation and the index of the values in xdata closest to x.
        """
        if not self._test_xdata_monotonic(xdata):
            raise ValueError("xdata should be monotonic")
        j_low = 0  # lowest index
        j_high = len(xdata) - 1  # highest index
        while (j_high - j_low) > 1:
```

```python
            j_middle = (j_high + j_low) >> 1
            if x >= xdata[j_middle]:
                j_low = j_middle
            else:
                j_high = j_middle
        return (
            max(0, min(len(xdata) - m, j_low - ((m - 2) >> 1))),
            j_low,
        )  # j_low now holds the midpoint. the higher the order m of interpolation, the more we have to go back to find the starting index.

    # -2 because when m=2, the middle point is the starting index. Because a higher order polynomial uses points on either side of x, we divide m by 2 (bitshift 1 to the right) when looking for the starting index.

    def _test_xdata_monotonic(self, xdata) -> bool:
        if xdata[1] > xdata[0]:  # xdata should be monotonically increasing
            return self._test_xdata_monotonically_increasing(xdata)
        if xdata[1] < xdata[0]:  # xdata should be monotonically decreasing
            return self._test_xdata_monotonically_decreasing(xdata)
        return False  # Catch case where first two values are equal

    def _test_xdata_monotonically_increasing(self, xdata) -> bool:
        return all(xdata[i] > xdata[i - 1] for i in range(1, len(xdata)))

    def _test_xdata_monotonically_decreasing(self, xdata) -> bool:
        return all(xdata[i] < xdata[i - 1] for i in range(1, len(xdata)))


class PolynomialInterpolater(BaseInterpolater):
    """Class to do polynomial interpolation using Neville's algorithm.
    m is the number of points to be used locally for interpolation. Therefore it is the order of the polynomial PLUS 1. i.e. m=2 means linear interpolation.
    We use a 1D vector to hold successive improvements (i.e. higher orders) of the polynomials.
    """

    def interpolate(self, x, xdata, ydata, m):
        starting_index, _ = super().find_starting_index_and_closest_index(x, xdata, m)
        interpolation_points = xdata[starting_index : starting_index + m]
        P = deepcopy(
            ydata[starting_index : starting_index + m]
        )  # 1D vector holding the polynomials
        for k in range(1, m):
            for i in range(m - k):
                j = i + k
                P[i] = (
                    (interpolation_points[j] - x) * P[i]
                    + (x - interpolation_points[i]) * P[i + 1]
                ) / (
                    interpolation_points[j] - interpolation_points[i]
                )  # e.g. (x_1-x)P_0 + (x-x_0)P_1 / x_1-x_0
            if k == (m - 1):  # last addition, get ready to save the error estimate
                self.error_estimate = np.abs(P[0] - P[1])
        return P[0]


def plot_part_a(
    x_data: np.ndarray,
    y_data: np.ndarray,
```

```python
    coeffs_c: np.ndarray,
    plots_dir: str = "Plots",
) -> None:
    """
    Ploting routine for part (a) results.

    Parameters
    ----------
    x_data : np.ndarray
        x-values.
    y_data : np.ndarray
        y-values.
    coeffs_c : np.ndarray
        Polynomial coefficients c.
    plots_dir : str
        Directory to save plots.

    Returns
    -------
    None
    """
    xx = np.linspace(x_data[0], x_data[-1], 1001)
    yy = evaluate_polynomial(coeffs_c, xx)
    y_at_data = evaluate_polynomial(coeffs_c, x_data)

    fig = plt.figure(figsize=(10, 10))
    gs = fig.add_gridspec(2, hspace=0, height_ratios=[2.0, 1.0])
    axs = gs.subplots(sharex=True, sharey=False)

    axs[0].plot(x_data, y_data, marker="o", linewidth=0)
    axs[0].plot(xx, yy, linewidth=3)
    axs[0].set_xlim(
        np.floor(xx[0]) - 0.01 * (xx[-1] - xx[0]),
        np.ceil(xx[-1]) + 0.01 * (xx[-1] - xx[0]),
    )
    axs[0].set_ylim(-400, 400)
    axs[0].set_ylabel("$y$")
    axs[0].legend(["data", "Via LU decomposition"], frameon=False, loc="lower left
    ")

    axs[1].set_ylim(1e-16, 1e1)
    axs[1].set_yscale("log")
    axs[1].set_ylabel(r"$|y - y_i|$")
    axs[1].set_xlabel("$x$")
    axs[1].plot(x_data, np.abs(y_data - y_at_data), linewidth=3)

    plt.savefig(os.path.join(plots_dir, "vandermonde_sol_2a.pdf"))
    plt.close()


def plot_part_b(
    x_data: np.ndarray,
    y_data: np.ndarray,
    plots_dir: str = "Plots",
) -> None:
    """Ploting routine for part (b) results.

    Parameters
    ----------
    x_data : np.ndarray
        x-values.
    y_data : np.ndarray
```

```
        y-values.
    plots_dir : str
        Directory to save plots.

    Returns
    -------
    None
    """
    xx = np.linspace(x_data[0], x_data[-1], 1001)
    yy = np.array(
        [
            PolynomialInterpolater().interpolate(x, x_data, y_data, m=len(x_data))
            for x in xx
        ],
        dtype=np.float64,
    )
    y_at_data = np.array(
        [
            PolynomialInterpolater().interpolate(x, x_data, y_data, m=len(x_data))
            for x in x_data
        ],
        dtype=np.float64,
    )

    fig = plt.figure(figsize=(10, 10))
    gs = fig.add_gridspec(2, hspace=0, height_ratios=[2.0, 1.0])
    axs = gs.subplots(sharex=True, sharey=False)

    axs[0].plot(x_data, y_data, marker="o", linewidth=0)
    axs[0].plot(xx, yy, linestyle="dashed", linewidth=3)
    axs[0].set_xlim(
        np.floor(xx[0]) - 0.01 * (xx[-1] - xx[0]),
        np.ceil(xx[-1]) + 0.01 * (xx[-1] - xx[0]),
    )
    axs[0].set_ylim(-400, 400)
    axs[0].set_ylabel("$y$")
    axs[0].legend(["data", "Via Neville's algorithm"], frameon=False, loc="lower left")

    axs[1].set_ylim(1e-16, 1e1)
    axs[1].set_yscale("log")
    axs[1].set_ylabel(r"$|y - y_i|$")
    axs[1].set_xlabel("$x$")
    axs[1].plot(x_data, np.abs(y_data - y_at_data), linestyle="dashed", linewidth=3)

    plt.savefig(os.path.join(plots_dir, "vandermonde_sol_2b.pdf"))
    plt.close()


def plot_part_c(
    x_data: np.ndarray,
    y_data: np.ndarray,
    coeffs_history: list[np.ndarray],
    iterations_num: list[int] = [0, 1, 10],
    plots_dir: str = "Plots",
) -> None:
    """Ploting routine for part (c) results.

    Parameters
    ----------
    x_data : np.ndarray
```

```python
        x-values.
    y_data : np.ndarray
        y-values.
    coeffs_history : list[np.ndarray]
        Coefficients per iteration.
    iterations_num : list[int]
        Iteration numbers to plot.
    plots_dir : str
        Directory to save plots.

    Returns
    -------
    None
    """
    linstyl = ["solid", "dashed", "dotted"]
    colors = ["tab:blue", "tab:orange", "tab:green"]

    xx = np.linspace(x_data[0], x_data[-1], 1001)

    fig = plt.figure(figsize=(10, 10))
    gs = fig.add_gridspec(2, hspace=0, height_ratios=[2.0, 1.0])
    axs = gs.subplots(sharex=True, sharey=False)

    axs[0].plot(x_data, y_data, marker="o", linewidth=0, color="black", label="data")

    for i, k in enumerate(iterations_num):
        if k >= len(coeffs_history):
            continue
        c = coeffs_history[k]
        yy = evaluate_polynomial(c, xx)
        y_at_data = evaluate_polynomial(c, x_data)
        diff = np.abs(y_at_data - y_data)

        axs[0].plot(
            xx,
            yy,
            linestyle=linstyl[i],
            color=colors[i],
            linewidth=3,
            label=f"Iteration {k}",
        )
        axs[1].plot(x_data, diff, linestyle=linstyl[i], color=colors[i], linewidth=3)

    axs[0].set_xlim(
        np.floor(xx[0]) - 0.01 * (xx[-1] - xx[0]),
        np.ceil(xx[-1]) + 0.01 * (xx[-1] - xx[0]),
    )
    axs[0].set_ylim(-400, 400)
    axs[0].set_ylabel("$y$")
    axs[0].legend(frameon=False, loc="lower left")

    axs[1].set_ylim(1e-16, 1e1)
    axs[1].set_yscale("log")
    axs[1].set_ylabel(r"$|y - y_i|$")
    axs[1].set_xlabel("$x$")

    plt.savefig(os.path.join(plots_dir, "vandermonde_sol_2c.pdf"))
    plt.close()


```

```python
def main():
    os.makedirs("Plots", exist_ok=True)
    x_data, y_data = load_data()
    V = construct_vandermonde_matrix(x_data)

    # I prefer a class implementation for the LU decomposition, based on the book "Numerical Recipes"
    # The constructor does the decomposition, the getter allows access to LU matrix itself
    # Since I will have to compare execution times in question 2d, I write two small wrappers here for 2a and 2c,
    # as I assume the LU decomposition itself should also be done the set number of times for comparison

    def vandermonde_solve_coefficients(V, y_data):
        ludcmp_instance = LUDecomposition(V)
        ludcmp_instance.solve(y_data)

    def vandermonde_solve_coefficients_with_iterative_improvement(
        V, y_data, iterations
    ):
        ludcmp_instance = LUDecomposition(V)
        ludcmp_instance.iterative_solve(y_data, iterations)

    # I also use a class implementation for the polynomial interpolator (which uses Neville's algorithm)

    # compute times
    number = 10

    t_a = (
        timeit.timeit(
            stmt=lambda: vandermonde_solve_coefficients(V, y_data),
            number=number,
        )
        / number
    )

    xx = np.linspace(x_data[0], x_data[-1], 1001)
    t_b = (
        timeit.timeit(
            stmt=lambda: np.array(
                [
                    PolynomialInterpolater().interpolate(
                        x, x_data, y_data, m=len(x_data)
                    )
                    for x in xx
                ],
                dtype=np.float64,
            ),
            number=number,
        )
        / number
    )

    t_c = (
        timeit.timeit(
            stmt=lambda: vandermonde_solve_coefficients_with_iterative_improvement(
                V, y_data, iterations=11
            ),
            number=number,
```

```
        )
        / number
    )

    # write all timing
    with open("Execution_times.txt", "w", encoding="utf-8") as f:
        f.write(f"\\item Execution time for part (a): {t_a:.5f} seconds\n")
        f.write(f"\\item Execution time for part (b): {t_b:.5f} seconds\n")
        f.write(f"\\item Execution time for part (c): {t_c:.5f} seconds\n")

    ludcmp_instance = LUDecomposition(V)
    c_a = ludcmp_instance.solve(y_data)
    plot_part_a(x_data, y_data, c_a)

    formatted_c = [f"{coef:.3e}" for coef in c_a]
    with open("Coefficients_output.txt", "w", encoding="utf-8") as f:
        for i, coef in enumerate(formatted_c):
            f.write(f"c$_{i + 1}$ = {coef}, ")

    plot_part_b(x_data, y_data)

    coeffs_history = ludcmp_instance.iterative_solve(y_data, iterations=11)
    plot_part_c(x_data, y_data, coeffs_history, iterations_num=[0, 1, 10])


if __name__ == "__main__":
    main()
```

2.6 Conclusions

As we can see from the plots in this section, Neville's algorithm is most accurate. The LU decomposition method starts to oscillate more and more as we increase our x values. This is likely due to the fact that the Vandermonde matrix becomes more and more ill-conditioned as we increase the x values, leading to larger numerical errors in the solution. I think this problem is amplified due to my method not implementing partial pivoting, which would help to reduce the numerical errors in the solution. The iterative improvement of the LU decomposition does not increase the accuracy. This is likely because the iterative solution suffers from the same problem as the initial solution.

However, in terms of speed, LU decomposition is much faster than Neville's algorithm in this case, roughly by a factor of 100. We also see that 10 extra iterations to improve the LU decomposition solution only doubles the execution time. Most of the execution time is spent on constructing the LU decomposition itself.